

Module 7 Structured Note: Finding Duplicates in a String

1. What this module is about

Module 7 teaches how to find **duplicate characters** in a C-style string.

A duplicate character means:

A character appears more than once in the same string.

Example:

```
char A[] = "finding";
```

The string is:

```
f i n d i n g
```

Here:

```
i appears 2 times  
n appears 2 times
```

So the duplicate letters are:

```
i and n
```

This module teaches two main methods:

1. Finding duplicates using **comparison**
2. Finding duplicates using a **hash table / counting array**

The main lesson is:

The comparison method is simple but slow. The hash table method is faster because it counts characters directly.

2. Required ideas from previous modules

Before Module 7, remember these ideas.

C-style strings end with `\0`

A C-style string is a character array ending with the null terminator:

```
'\0'
```

Example:

```
char A[] = "finding";
```

Memory view:

```
Index: 0 1 2 3 4 5 6 7  
Value: f i n d i n g \0
```

The visible string is:

```
finding
```

The `\0` is not part of the word. It only tells C where the string ends.

So most string loops use this pattern:

```
for (i = 0; A[i] != '\0'; i++) {  
    // process A[i]  
}
```

Meaning:

1. Start at index `0`.
 2. Check one character at a time.
 3. Stop when `\0` is found.
-

3. Meaning of duplicates

A duplicate means a character appears again after its first occurrence.

Example:

finding

Character list:

f i n d i n g

Check each character:

Character	Appears how many times?	Duplicate?
f	1	No
i	2	Yes
n	2	Yes
d	1	No
g	1	No

So the answer is:

i and n are duplicates

Part A: Method 1 — Finding duplicates by comparison

4. Main idea of the comparison method

The comparison method checks each character against the characters that come after it.

For example, in:

finding

Start with **f**:

f i n d i n g
^

Compare **f** with the rest:

i n d i n g

No more **f**, so **f** is not duplicate.

Next, check **i**:

f i n d i n g
^

Compare **i** with the rest:

n d i n g

Another **i** is found, so **i** is duplicate.

Next, check **n**:

f i n d i n g
^

Another **n** is found later, so **n** is duplicate.

5. Why this method needs two loops

To compare each character with the characters after it, we need:

- one loop to pick the current character
- another loop to search the rest of the string

The outer loop chooses a character:

```
for (i = 0; A[i] != '\0'; i++)
```

The inner loop checks the characters after it:

```
for (j = i + 1; A[j] != '\0'; j++)
```

So the structure looks like this:

```
for (i = 0; A[i] != '\0'; i++) {  
    for (j = i + 1; A[j] != '\0'; j++) {  
        // compare A[i] and A[j]  
    }  
}
```

This is called a **nested loop** because one loop is inside another loop.

6. Counting matches in the comparison method

When checking a character, we use a counter.

Example:

```
count = 1;
```

Why does it start at `1`?

Because the current character itself already appears once.

Example:

```
finding  
 ^
```

When checking the first `i`, we already know there is one `i`. So:

```
count = 1
```

If we find another `i`, we increase the count:

```
count++;
```

At the end:

```
if (count > 1)
```

then the character is duplicate.

7. Why we mark already-counted characters

Suppose we find that `i` is duplicate in:

```
finding
```

The string has two `i` characters:

```
f i n d i n g
  ^     ^
```

After counting the first `i`, we should not count the second `i` again later.

So we mark the second `i` as already counted.

One beginner method is:

```
A[j] = '0';
```

This means:

This repeated character has already been counted. Ignore it later.

Important detail:

'0'

is the character zero, not the number zero.

It has ASCII value 48.

This marker is only safe if the original string does not need the character '0' as real input.

8. Code: duplicate detection using comparison

```
#include <stdio.h>

int main() {
    char A[] = "finding";
    int i, j, count;

    for (i = 0; A[i] != '\0'; i++) {
        count = 1;

        if (A[i] != '0') {
            for (j = i + 1; A[j] != '\0'; j++) {
                if (A[i] == A[j]) {
                    count++;
                    A[j] = '0';
                }
            }

            if (count > 1) {
                printf("%c appears %d times\n", A[i], count);
            }
        }
    }

    return 0;
}
```

Output:

```
i appears 2 times
n appears 2 times
```

9. Dry run: comparison method for "finding"

Original string:

```
f i n d i n g
```

Step 1: Check **f**

```
f i n d i n g  
^
```

Compare **f** with:

```
i n d i n g
```

No match found.

Result:

```
f is not duplicate
```

Step 2: Check first **i**

```
f i n d i n g  
      ^
```

Compare **i** with:

```
n d i n g
```

Another **i** is found.

So:

```
count = 2
```

Mark the second **i**:


```
f i n d 0 n g
```

Print:

```
i appears 2 times
```

Step 3: Check first `n`

```
f i n d 0 n g
      ^
```

Another `n` is found later.

So:

```
count = 2
```

Mark the second `n`:

```
f i n d 0 0 g
```

Print:

```
n appears 2 times
```

Step 4: Continue checking

Characters `d` and `g` appear once.

The marked characters `0` are ignored.

Final output:

```
i appears 2 times
n appears 2 times
```

10. Complexity of the comparison method

The comparison method uses a loop inside another loop.

For each character, it may check many other characters.

So the time complexity is:

$O(N^2)$

The extra space complexity is:

$O(1)$

Why?

Because it only uses a few variables:

```
int i, j, count;
```

It does not create another large array.

Summary of Method 1

Feature	Value
Main idea	Compare each character with the rest
Time complexity	$O(N^2)$
Extra space	$O(1)$
Easy to understand?	Yes
Good for long strings?	No

Part B: Method 2 — Finding duplicates using a hash table

11. Main idea of the hash table method

The comparison method repeatedly searches the string.

The hash table method avoids repeated searching.

Instead of asking:

Does this character appear again later?

we ask:

How many times does each character appear?

For lowercase English letters, there are 26 possible letters:

```
a b c d e f g h i j k l m n o p q r s t u v w x y z
```

So we create an integer array with 26 slots:

```
int H[26] = {0};
```

This array stores letter counts.

12. Meaning of `H[26]`

The array:

```
int H[26] = {0};
```

has 26 positions:

```
H[0], H[1], H[2], ..., H[25]
```

Each position represents one lowercase letter.

```
H[0] -> count of a
H[1] -> count of b
H[2] -> count of c
H[3] -> count of d
H[4] -> count of e
H[5] -> count of f
...
H[25] -> count of z
```

At the start, all counts are zero:

```
H[0] = 0
H[1] = 0
H[2] = 0
...
H[25] = 0
```

This means:

We have not counted any letters yet.

13. ASCII mapping: converting a letter to an index

Lowercase letters have ASCII values from 97 to 122.

```
'a' = 97
'b' = 98
'c' = 99
...
'z' = 122
```

But array indexes start from 0.

So we need to convert letters into indexes.

The rule is:

```
index = character - 'a';
```

This is the same as:

```
index = character - 97;
```

But this version is easier to read:

```
index = character - 'a';
```

Examples:

```
'a' - 'a' = 0  
'b' - 'a' = 1  
'c' - 'a' = 2  
'd' - 'a' = 3  
'f' - 'a' = 5  
'i' - 'a' = 8  
'n' - 'a' = 13  
'z' - 'a' = 25
```

So when we see a letter, we can update its count like this:

```
H[A[i] - 'a']++;
```

Meaning:

Go to the correct letter slot and increase its count.

14. Code: duplicate detection using hash table

```
#include <stdio.h>  
  
int main() {  
    char A[] = "finding";  
    int H[26] = {0};  
    int i;  
  
    for (i = 0; A[i] != '\0'; i++) {  
        H[A[i] - 'a']++;  
    }  
  
    for (i = 0; i < 26; i++) {  
        if (H[i] > 1) {
```

```
        printf("%c appears %d times\n", i + 'a', H[i]);
    }
}

return 0;
}
```

Output:

```
i appears 2 times
n appears 2 times
```

15. Understanding the first loop

The first loop scans the string:

```
for (i = 0; A[i] != '\0'; i++) {
    H[A[i] - 'a']++;
}
```

This means:

1. Look at the current character.
2. Convert it into an index using `A[i] - 'a'`.
3. Increase the count in `H`.
4. Move to the next character.
5. Stop at `\0`.

Example:

```
A[i] = 'f'
```

Then:

```
'f' - 'a' = 5
```

So:

```
H[5]++;
```

This increases the count of `f`.

16. Understanding the second loop

After counting all letters, we scan the hash table:

```
for (i = 0; i < 26; i++) {  
    if (H[i] > 1) {  
        printf("%c appears %d times\n", i + 'a', H[i]);  
    }  
}
```

This means:

1. Check every letter count from `a` to `z`.
 2. If a count is greater than `1`, that letter is duplicate.
 3. Print the letter and its count.
-

17. Why `i + 'a'` prints the letter

When scanning `H`, the variable `i` is an index.

It is not directly a character.

Example:

```
i = 0 means a  
i = 1 means b  
i = 2 means c  
i = 8 means i  
i = 13 means n
```

To convert the index back into a letter, we use:

```
i + 'a'
```

Example:

```
8 + 'a' = 8 + 97 = 105
```

ASCII value `105` is:

```
i
```

So:

```
printf("%c", i + 'a');
```

prints the letter.

18. Dry run: hash table method for "finding"

String:

```
f i n d i n g
```

Initial hash table:

```
All values are 0
```

Now scan each character.

Character	Index calculation	Hash update
f	f - a = 5	H[5] becomes 1
i	i - a = 8	H[8] becomes 1
n	n - a = 13	H[13] becomes 1
d	d - a = 3	H[3] becomes 1
i	i - a = 8	H[8] becomes 2
n	n - a = 13	H[13] becomes 2
g	g - a = 6	H[6] becomes 1

Final important hash table values:


```
H[3]  = 1  -> d
H[5]  = 1  -> f
H[6]  = 1  -> g
H[8]  = 2  -> i
H[13] = 2  -> n
```

Now scan **H**.

```
H[8] > 1  -> i is duplicate
H[13] > 1  -> n is duplicate
```

Final output:

```
i appears 2 times
n appears 2 times
```

19. Complexity of the hash table method

The hash table method has two parts.

Part 1: Scan the string

```
for (i = 0; A[i] != '\0'; i++)
```

If the string has **N** characters, this takes:

$O(N)$

Part 2: Scan the hash table

```
for (i = 0; i < 26; i++)
```

This takes:

$O(26)$

So total time is:

$O(N + 26)$

Since `26` is constant, we simplify this to:

$O(N)$

Space complexity

The hash table uses:

```
int H[26]
```

This array always has 26 slots.

It does not become larger when the input string becomes larger.

So for lowercase English letters, the extra space is:

$O(1)$

Summary of Method 2

Feature	Value
Main idea	Count each letter using <code>H[26]</code>
Time complexity	<code>O(N)</code>
Extra space	<code>O(1)</code> for fixed lowercase letters
Easy to understand?	Medium
Good for long strings?	Yes

Part C: Important limitations and improvements

20. Limitation of `H[26]`

This code:

```
H[A[i] - 'a']++;
```

assumes that every character is a lowercase letter from `a` to `z`.

It works for:

```
finding  
programming  
banana
```

But it does not safely handle:

```
uppercase letters  
numbers  
spaces  
symbols
```

Example:

```
char A[] = "Find123";
```

The character `F` is uppercase.

So:

```
'F' - 'a'
```

will not give an index between `0` and `25`.

That can lead to invalid array access.

So for Module 7, the basic `H[26]` method is meant for lowercase English letters only.

21. Safer lowercase-only version

A safer version checks whether the character is lowercase before using it as an index.

```

#include <stdio.h>

int main() {
    char A[] = "finding";
    int H[26] = {0};
    int i;

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 'a' && A[i] <= 'z') {
            H[A[i] - 'a']++;
        }
    }

    for (i = 0; i < 26; i++) {
        if (H[i] > 1) {
            printf("%c appears %d times\n", i + 'a', H[i]);
        }
    }

    return 0;
}

```

This version ignores characters that are not lowercase letters.

22. How to handle uppercase letters

There are two simple options.

Option 1: Convert uppercase to lowercase first

Example:

Finding

Convert it to:

finding

Then use `H[26]`.

Basic conversion rule:

```
if (A[i] >= 'A' && A[i] <= 'Z') {
    A[i] = A[i] + 32;
}
```

Then count using:

```
H[A[i] - 'a']++;
```

Option 2: Use a larger ASCII table

Instead of only 26 lowercase letters, use 128 ASCII slots:

```
int H[128] = {0};
```

Then you can count characters by their ASCII values directly:

```
H[A[i]]++;
```

This can handle:

```
uppercase letters
lowercase letters
digits
spaces
symbols
```

But the Module 7 basic method focuses on lowercase letters with `H[26]`.

Part D: Comparing both methods

23. Method comparison table

Feature	Comparison method	Hash table method
Main idea	Compare each character with the rest	Count each character
Main structure	Nested loops	Counting array
Time complexity	<code>O(N²)</code>	<code>O(N)</code>

Feature	Comparison method	Hash table method
Extra space	$O(1)$	$O(1)$ for fixed $H[26]$
Modifies original string?	Yes, if marking is used	No
Good for learning?	Yes	Yes
Good for large strings?	No	Yes

24. When to use which method

Use the comparison method when:

- you are learning the basic idea
- the string is very small
- you want to understand direct character comparison

Use the hash table method when:

- you want a faster solution
- the string may be large
- you need actual counts of repeated letters

In practice, the hash table method is usually better.

Part E: Common mistakes

25. Mistake 1: Forgetting to stop at `\0`

Wrong:

```
for (i = 0; i < 100; i++)
```

Better:

```
for (i = 0; A[i] != '\0'; i++)
```

Why?

Because the string may end before index `100`.

The real end of a C-style string is marked by `\0`.

26. Mistake 2: Not marking counted duplicates in Method 1

If you do not mark duplicates, the same character may be counted again later.

Example:

```
finding
```

After counting the first `i`, the second `i` may be counted again unless it is marked.

That is why Method 1 may use:

```
A[j] = '0';
```

27. Mistake 3: Confusing `'0'` and `0`

These are different:

```
'0'
```

and

```
0
```

`'0'` is the character zero.

Its ASCII value is `48`.

`0` is the number zero.

In Method 1, when we write:

```
A[j] = '0';
```

we are storing the character `'0'`, not the null terminator.

Do not confuse it with:

```
'\0'
```

The null terminator `\0` means end of string.

28. Mistake 4: Using `H[A[i] - 'a']` without checking the character

This is safe only if `A[i]` is lowercase from `a` to `z`.

Risky:

```
H[A[i] - 'a']++;
```

Safer:

```
if (A[i] >= 'a' && A[i] <= 'z') {  
    H[A[i] - 'a']++;  
}
```

Without the check, uppercase letters, digits, spaces, or symbols can create invalid indexes.

29. Mistake 5: Thinking `H[26]` is `O(N)` space

`H[26]` has a fixed size.

It always has 26 slots.

It does not matter whether the string length is:

```
5  
50  
500  
5000
```

The hash table still has 26 positions.

So the extra space is:

`O(1)`

not `O(N)`.

Part F: Main code patterns to remember

30. Pattern 1: Scan a string

```
for (i = 0; A[i] != '\0'; i++) {  
    // process A[i]  
}
```

31. Pattern 2: Compare each character with the rest

```
for (i = 0; A[i] != '\0'; i++) {  
    for (j = i + 1; A[j] != '\0'; j++) {  
        if (A[i] == A[j]) {  
            // duplicate found  
        }  
    }  
}
```

32. Pattern 3: Create a hash table for lowercase letters

```
int H[26] = {0};
```

33. Pattern 4: Convert lowercase letter to index

```
A[i] - 'a'
```

Examples:

```
a -> 0
b -> 1
f -> 5
i -> 8
n -> 13
z -> 25
```

34. Pattern 5: Count each lowercase letter

```
for (i = 0; A[i] != '\0'; i++) {
    H[A[i] - 'a']++;
}
```

Safer version:

```
for (i = 0; A[i] != '\0'; i++) {
    if (A[i] >= 'a' && A[i] <= 'z') {
        H[A[i] - 'a']++;
    }
}
```

35. Pattern 6: Print duplicate letters

```
for (i = 0; i < 26; i++) {
    if (H[i] > 1) {
        printf("%c appears %d times\n", i + 'a', H[i]);
    }
}
```

Part G: Full combined examples

36. Full program: comparison method

```
#include <stdio.h>

int main() {
```

```

char A[] = "finding";
int i, j, count;

for (i = 0; A[i] != '\0'; i++) {
    count = 1;

    if (A[i] != '0') {
        for (j = i + 1; A[j] != '\0'; j++) {
            if (A[i] == A[j]) {
                count++;
                A[j] = '0';
            }
        }

        if (count > 1) {
            printf("%c appears %d times\n", A[i], count);
        }
    }
}

return 0;
}

```

Output:

```

i appears 2 times
n appears 2 times

```

37. Full program: hash table method

```

#include <stdio.h>

int main() {
    char A[] = "finding";
    int H[26] = {0};
    int i;

    for (i = 0; A[i] != '\0'; i++) {
        H[A[i] - 'a']++;
    }

    for (i = 0; i < 26; i++) {
        if (H[i] > 1) {

```

```

        printf("%c appears %d times\n", i + 'a', H[i]);
    }
}

return 0;
}

```

Output:

```

i appears 2 times
n appears 2 times

```

38. Full program: safer lowercase-only hash table method

```

#include <stdio.h>

int main() {
    char A[] = "finding";
    int H[26] = {0};
    int i;

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 'a' && A[i] <= 'z') {
            H[A[i] - 'a']++;
        }
    }

    for (i = 0; i < 26; i++) {
        if (H[i] > 1) {
            printf("%c appears %d times\n", i + 'a', H[i]);
        }
    }

    return 0;
}

```

Output:

```

i appears 2 times
n appears 2 times

```

Part H: Practice questions

39. Practice 1

Find duplicates in:

```
finding
```

Expected answer:

```
i appears 2 times  
n appears 2 times
```

40. Practice 2

For the string:

```
finding
```

Write the hash table indexes used.

Answer:

```
f -> 5  
i -> 8  
n -> 13  
d -> 3  
i -> 8  
n -> 13  
g -> 6
```

41. Practice 3

Why is `H[26]` enough for lowercase letters?

Answer:

Because lowercase English has 26 letters:

a to z

Each letter gets one slot in the hash table.

42. Practice 4

Why is `H[26]` not enough for uppercase plus lowercase letters without adjustment?

Answer:

Because uppercase and lowercase letters are different characters in ASCII.

Example:

'A' and 'a' are different

If we only use `H[26]` with `A[i] - 'a'`, uppercase letters do not map safely to indexes `0` to `25`.

43. Practice 5

Find duplicates in:

programming

Hint:

Count each letter using `H[26]`.

Expected repeated letters:

r appears 2 times
g appears 2 times
m appears 2 times

44. Practice 6

Explain why the comparison method is slower than the hash table method.

Answer:

The comparison method uses nested loops, so it may compare many pairs of characters.

That gives:

$O(N^2)$

The hash table method scans the string once and then scans 26 fixed positions.

That gives:

$O(N)$

Part I: Final summary

45. Key points to remember

1. A duplicate character appears more than once.
 2. Module 7 uses the same string traversal pattern: stop at `\0`.
 3. The comparison method checks each character against the rest of the string.
 4. The comparison method uses nested loops.
 5. The comparison method may mark already-counted duplicates using `'0'`.
 6. The comparison method takes $O(N^2)$ time.
 7. The hash table method uses an array `H[26]`.
 8. `H[26]` stores counts for lowercase letters `a` to `z`.
 9. A lowercase character is mapped to an index using `A[i] - 'a'`.
 10. An index is converted back to a character using `i + 'a'`.
 11. The hash table method takes $O(N + 26)$, which simplifies to $O(N)$.
 12. The hash table method uses $O(1)$ extra space because `H[26]` is fixed size.
 13. The basic hash table method assumes lowercase letters only.
 14. Use range checks to avoid invalid indexes.
 15. For uppercase letters, either convert to lowercase first or use a larger ASCII table.
-

46. Final mental model

For the comparison method:

```
Pick one character.  
Compare it with all later characters.  
Count matches.  
Mark repeated characters so they are not counted again.  
Print the character if count is greater than 1.
```

For the hash table method:

```
Create H[26].  
Each letter has one slot.  
Scan the string once.  
Increase the count for each letter.  
Scan H.  
Print letters whose count is greater than 1.
```

47. Module 7 checkpoint

You are ready for Module 8 if you can do these without help:

1. Define what a duplicate character is.
2. Find duplicate letters in `finding`.
3. Explain the comparison method.
4. Write the nested-loop duplicate detection code.
5. Explain why already-counted duplicates may need to be marked.
6. State the comparison method time complexity as $O(N^2)$.
7. Create `int H[26] = {0};`.
8. Explain what each slot in `H[26]` means.
9. Convert lowercase letters to indexes using `A[i] - 'a'`.
10. Convert indexes back to letters using `i + 'a'`.
11. Dry-run the hash table method for `finding`.
12. Explain why the hash table method is $O(N)$.
13. Explain why `H[26]` is $O(1)$ extra space.
14. Explain why the basic hash table code assumes lowercase letters.
15. Write a safer version using lowercase range checks.

Final takeaway:

Module 7 teaches how to find repeated characters. The simple comparison method works, but it is slow. The hash table method is better because it counts each letter directly and finds duplicates in linear time.